

COPPER paper source fallback render

```
\documentclass[10pt]{article}
\usepackage[margin=0.75in]{geometry}
\usepackage{url}
\usepackage{graphicx}
```

```
\title{COPPER: Committed Pointer-Provenance Prefetching}
\author{Anonymous Artifact Submission}
\date{}
```

```
\begin{document}
\maketitle
```

```
\begin{abstract}
```

Data-derived prefetchers can reduce pointer-chasing latency, but they can also turn address-shaped data into prefetch authority before the program has proved that the data is a pointer. COPPER addresses that authority problem with committed pointer-provenance prefetching: it records pointer-source evidence only after architectural commit, invalidates stale or mismatched evidence, and gates later data-derived prefetch issue on that committed evidence. This submission frames COPPER as a scoped reproducible hardware mechanism study evaluated with CI-proven RTL simulation, raw gem5 full-system provenance where retained, validated imported gem5 ARM-system summaries where PASS, independent simulator evidence, accepted-core-wrapper/scoped PicoRV32 tiny-SoC full-core FPGA mapped PPA, and scoped tool-estimated/proxy power. The supported claim is deliberately narrow: COPPER is an auditable mechanism with reproducible evidence at the stated levels. It does not claim state-of-the-art performance, production readiness, a complete clone-local gem5 campaign, production ARM/OoO integration, production-core mapped timing, measured silicon power efficiency, ASIC signoff, foundry signoff, or silicon signoff.

```
\end{abstract}
```

```
\section{Introduction}
```

Pointer-heavy programs expose memory latency that ordinary address-stream prefetchers can miss. Data-derived prefetching can help by following loaded values, but the same behavior is risky when a value merely resembles an address. COPPER changes the authority rule: a prefetch candidate needs committed pointer-source evidence before issue. The mechanism is meant to filter unsafe or stale candidates while preserving the useful subset of pointer-derived prefetches.

This paper makes a scoped contribution. COPPER is evaluated as a reproducible hardware mechanism, not as a silicon product or a production processor feature. The evidence consists of executable model checks, deterministic cycle-model and core-integrated rows, source-backed independent simulator rows, validated gem5 ARM-system summaries and retained raw gem5 provenance where available, CI-proven RTL unit simulation, accepted-core-wrapper/scoped PicoRV32 tiny-SoC full-core mapped FPGA PPA, FPGA tool-estimated power, and proxy energy. The artifact keeps these evidence levels separate and keeps negative rows visible.

This paper does not claim silicon-proven status, ASIC/foundry signoff, measured silicon power, production ARM/OoO/TLB/cache/coherence integration, state-of-the-art silicon efficiency, universal speedup, production readiness, or full silicon PPA. The claim ledger and audits are part of the submission: if the paper drifts into unsupported language, the artifact should fail.

```
\section{Motivation}
```

```
\begin{figure}[t]
```

```
\centering
```

```
\fbox{\begin{minipage}{\linewidth}\centering
```

Demand execution uses a loaded pointer after commit. COPPER records that source word as eligible. Later data-derived prefetch issue checks the same source identity, value evidence, domain, and permission path before acting.

```
\end{minipage}}
```

```
\caption{Pointer-chasing motivation.}
```

```
\label{fig:motivation}
```

```
\end{figure}
```

The motivating failure mode is not ordinary speculation alone. It is that a prefetcher may treat address-shaped data as authority, a concern sharpened by recent data-memory-dependent prefetcher attacks \cite{augury,gofetch}. COPPER instead treats committed pointer use as authority.

```
\section{Background And Related Work}
```

Prior work covers next-line, stream, stride, indirect, pointer-chase, runahead, helper-thread, and dependence-guided prefetching

\cite{jouppi1990improving,chen1995effective,baer1991effective,roth1998dependence,srinath2007feedback}, as well as metadata and capability mechanisms \cite{cheri}. COPPER's distinction is narrower: committed source-word proof gates data-derived prefetch issue. The repository matrix records what COPPER does not claim.

\section{COPPER Design}

\begin{figure}[t]

\centering

\fbox{\begin{minipage}{\linewidth}\centering

Commit path: observe pointer-source use. Provenance table: retain clean source evidence. Prefetch path: require source match, target permission, and valid translation context. Invalidation path: clear source evidence on writes and coherence events.

\end{minipage}}

\caption{COPPER architecture.}

\label{fig:architecture}

\end{figure}

\begin{figure}[t]

\centering

\fbox{\begin{minipage}{\linewidth}\centering

Load source word, use as demand address, commit, record proof, later prefetch candidate, check proof, issue or block.

\end{minipage}}

\caption{Committed provenance update timeline.}

\label{fig:timeline}

\end{figure}

COPPER has three core rules: proof creation occurs after committed architectural evidence; writes, coherence updates, failed permissions, and context mismatch block or destroy proof; recursive issue does not gain authority merely because a line arrived by prefetch.

\section{Implementation}

The artifact contains a Python model, a trace-driven evaluation harness, SystemVerilog RTL units, C/C++ workload sources, reproduction scripts, and summary parsers. The open-source RTL smoke target is

\texttt{copper_prefetch_unit_open}; larger local Vivado summaries are treated as existing evidence rather than portable rerun proof. The gem5 evidence is validated through retained summaries and provenance files where available \cite{binkert2011gem5}; the scoped full-core hardware harness uses PicoRV32 rather than a production ARM/OoO target \cite{wolf2019picorv32}.

\section{Methodology}

\begin{table}[t]

\centering

\caption{Benchmarks}

\begin{tabular}{lll}

\hline

Class & Examples & Evidence file \\\

\hline

Pointer-heavy & linked list, tree, hash, graph, Patricia & workload_build.csv; independent_sim_performance.csv

\\

Controls & array, matrix, compute, random access & workload_build.csv; independent_sim_performance.csv \\\

Stress & short chains, long chains, mixed, noisy, branchy & workload_build.csv; independent_sim_performance.csv

\\

\hline

\end{tabular}

\end{table}

\begin{table}[t]

\centering

\caption{Configurations And Baselines}

\begin{tabular}{lll}

\hline

Config & Role & Evidence file \\\

\hline

no_prefetch & reference & cycle_performance.csv \\\

next_line & spatial baseline & cycle_performance.csv \\\

stride & regular-stream baseline & cycle_performance.csv \\\

simple_pointer_chase & unsafe content-derived baseline & cycle_performance.csv \\\

copper & committed-provenance policy & cycle_performance.csv \\\

\hline

\end{tabular}

\end{table}

The normalized CSVs report per-workload rows rather than hiding negative results inside averages. Evidence levels are explicit: \texttt{model} rows come from the executable policy model, \texttt{cycle_model} rows come from a

deterministic memory-system model with hit/miss latency, memory latency, outstanding prefetches, queue drops, lateness, and demand/prefetch traffic accounting. \texttt{core_integrated} rows add a deterministic core envelope with fetch/issue width, reorder-window pressure, load-queue pressure, branch penalties, and memory-system timing. \texttt{independent_sim} rows execute the source-built C workload driver for checksums and then use a separate trace/event cache simulator that does not import the cycle-model or core-integrated harness. \texttt{gem5_full_system} rows are imported from validated ARM-system summaries only when the compared group has a no-prefetch baseline, a COPPER-family row, matching checksums, zero return codes, and positive tick counts. \texttt{gem5_statistical_summary.csv} reports imported-summary confidence intervals where repeated samples exist and marks single-sample rows explicitly. This paper does not claim a clone-local rerun of every raw simulator run. None of these rows claim complete CPU integration.

Evaluation

Speedup rows are generated in \texttt{research/results/performance.csv}, \texttt{research/results/cycle_performance.csv}, \texttt{research/results/core_integrated_performance.csv}, and \texttt{research/results/independent_sim_performance.csv}. The paper does not collapse them into a single broad win claim.

Speedup by benchmark.

fig:speedup

Issued, useful, useless, accuracy, coverage, lateness, and queue-drop fields are generated in \texttt{research/results/prefetch_metrics.csv}, \texttt{research/results/cycle_prefetch_metrics.csv}, \texttt{research/results/core_integrated_prefetch_metrics.csv}, and \texttt{research/results/independent_sim_prefetch_metrics.csv}.

Prefetch accuracy and coverage.

fig:prefetch

Demand-load, prefetch-load, total-request, and traffic-overhead fields are generated in \texttt{research/results/memory_traffic.csv}, \texttt{research/results/cycle_memory_traffic.csv}, \texttt{research/results/core_integrated_memory_traffic.csv}, and \texttt{research/results/independent_sim_memory_traffic.csv}.

Traffic overhead.

fig:traffic

Current evidence supports selective, per-workload discussion. Several cycle-model, core-integrated, and independent-sim rows show COPPER behind the best baseline; those regressions remain in the CSVs and block a universal speedup claim.

Ablation And Sensitivity

Ablation and sensitivity rows are generated in \texttt{research/results/ablation.csv} and \texttt{research/results/sensitivity.csv}; the rows include evidence-level labels.

Ablation and sensitivity.

fig:ablation

The cycle-model ablations isolate no provenance, speculative provenance, committed-only proof, confidence, queue filtering, and full COPPER. Queue size, confidence threshold, chain depth, distance, table size, and memory latency are varied in sensitivity rows. These are cycle-model counters and should not be described as gem5 counters.

Hardware Cost

```

\centering
\caption{Hardware Cost}
\begin{tabular}{|l|}
\hline
Evidence & Scope & File \\
\hline
CI RTL simulation & open-source unit test & rtl\_simulation.csv \\
Yosys & generic unit resource context & synthesis.csv \\
nextpnr & mapped unit resource context when available & synthesis.csv \\
Overhead & matched unit rows from same flow & synthesis\_overhead.csv \\
Near-core stub & matched generic near-core-stub rows when Yosys runs & fullcore\_synthesis\_overhead.csv \\
PicoRV32 core-wrapper & matched generic accepted core-wrapper rows when Yosys runs & fullcore\_synthesis\_overhead.csv \\
PicoRV32 tiny-SoC full-core & matched full-core rows when Yosys or mapped flows run & fullcore\_synthesis\_overhead.csv; mapped\_ppa.csv \\
Mapped PPA & matched near-core-stub, PicoRV32 core-wrapper, PicoRV32 tiny-SoC full-core, or unit rows only when place-and-route succeeds & mapped\_ppa.csv; mapped\_ppa\_overhead.csv \\
Production-core integration & not claimed & top\_tier\_gate\_status.csv \\
\hline
\end{tabular}
\end{table}

```

The artifact supports unit-level hardware plausibility, matched near-core-stub resource-overhead rows, matched PicoRV32 accepted-core-wrapper rows, and matched PicoRV32 tiny-SoC full-core rows when the generated ledgers mark them PASS. The near-core stub is not a full CPU, the accepted wrapper is not full-core, and the PicoRV32 tiny-SoC full-core harness is not a production ARM/OoO integration. Generic Yosys rows do not provide mapped timing, Fmax, ASIC area, or measured power. Mapped timing may be discussed only when \texttt{mapped_ppa.csv} contains matched PASS rows from nextpnr, Vivado, or OpenROAD.

```

\section{Energy Proxy}
\begin{table}[t]
\centering
\caption{Energy/Power Evidence}
\begin{tabular}{|l|}
\hline
Evidence & Scope & File \\
\hline
Memory traffic proxy & assumption-based, not measured & energy\_proxy.csv \\
ASIC Liberty tool-power index & Nangate45 standard-cell estimate when indexed PASS & ASIC\_power.csv; power\_report\_index.csv \\
OpenROAD post-route power index & Nangate45 post-route tool estimate with indexed final DEF/SPEF/netlist when PASS & openroad\_postroute\_power.csv; power\_report\_index.csv \\
FPGA tool-power index & Vivado FPGA tool estimate when indexed PASS & power\_report\_index.csv; mapped\_ppa.csv \\
Activity proxy & McPAT proxy when indexed PASS & power\_report\_index.csv; copper\_mcpat\_sensitivity\_20260618.csv \\
\hline
Summary & proxy overhead statistics & energy\_summary.csv \\
\hline
\end{tabular}
\end{table}

```

Energy rows use explicit assumptions recorded in \texttt{energy_proxy.csv}. When \texttt{power_report_index.csv} marks \texttt{openroad_postroute_tool_estimate} PASS, the row is a Nangate45 OpenROAD-flow-scripts post-route estimate with indexed final DEF/SPEF/netlist artifacts, not silicon measurement or foundry signoff. When \texttt{ASIC_liberty_tool_estimate} is PASS, the row is a Nangate45 standard-cell Liberty estimate from OpenSTA/OpenROAD, not silicon measurement or post-route signoff with extracted parasitics. When \texttt{fpga_tool_estimate} is PASS, the row is Vivado \texttt{report_power} for the mapped FPGA target, not silicon or ASIC signoff. When \texttt{proxy_activity} is PASS, the activity proxy is the fixed-architecture McPAT sensitivity run driven by measured gem5 ROI counters. This supports scoped tool-power and proxy/model energy discussion, not foundry-signoff or silicon power-efficiency claims.

```

\section{Limitations}
\begin{table}[t]
\centering
\caption{Limitations}
\begin{tabular}{|l|}
\hline
Limitation & Consequence \\
\hline
Gem5 rows are validated imported ARM-system summaries & Gem5 validation is scoped to imported PASS rows
\end{tabular}
\end{table}

```

Independent simulator is trace/event level & It is not a replacement for a broad gem5 campaign \\
PicoRV32 tiny-SoC is the scoped full-core target & Production ARM/OoO claims remain blocked \\
Power is tool-estimate/model based & Signoff and silicon power-efficiency claims remain blocked \\
External gem5 and Vivado setup & Large external-tool reruns are not clone-local \\
\\hline

\\end{tabular}
\\end{table}

\\section{Artifact Evidence Gates}

\\begin{table}[t]

\\centering

\\caption{Artifact And Evidence Gates}

\\begin{tabular}{lll}

\\hline

Gate class & Status source & Interpretation \\
\\hline

CI RTL evidence & rtl_simulation.csv & GitHub Actions unit simulation \\
Cycle-model evidence & cycle_performance.csv & deterministic memory-system timing model \\
Core-integrated evidence & core_integrated_performance.csv & deterministic core-envelope model \\
Independent simulator & independent_sim_performance.csv & source-backed trace/event simulator \\
Gem5 validated summaries & gem5_validation.csv; gem5_performance.csv; gem5_statistical_summary.csv & imported
ARM-system rows and imported-summary statistics when PASS \\
Hardware cost & synthesis_overhead.csv; fullcore_synthesis_overhead.csv; mapped_ppa.csv & matched unit, near-
core-stub, PicoRV32 core-wrapper, or PicoRV32 tiny-SoC full-core resources, plus mapped timing only when PASS \\
Energy proxy & energy_proxy.csv; openroad_postroute_power.csv; asic_power.csv; power_report_index.csv &
assumption-based proxy plus OpenROAD post-route, ASIC Liberty, or FPGA tool estimates when indexed PASS \\
Package & artifact_manifest.csv & included and excluded files \\
\\hline

\\end{tabular}

\\end{table}

\\section{Conclusion}

COPPER is best framed as a committed-provenance authority mechanism for data-derived prefetch issue. The artifact supports that claim with a CI-proven open-source path, deterministic cycle-model and core-integrated evidence, source-backed independent-simulator rows, validated imported gem5 ARM-system rows with imported-summary statistics, matched unit-level, near-core-stub, PicoRV32 accepted-core-wrapper, and scoped PicoRV32 tiny-SoC full-core FPGA PPA paths, FPGA tool-estimated power where indexed PASS, proxy energy rows, a claim ledger, audits, and a package manifest. The honest next step for a broader architecture submission is a final raw-rerunnable gem5 or comparable external core validation matrix plus signoff-calibrated or silicon-measured power evidence without changing the scoped claims in this artifact.

\\bibliographystyle{plain}

\\bibliography{references}

\\end{document}